

Modelo Dinámico de Estabilidad Térmica y Rendimiento en Computadores de Propósito General

David Alexander Salazar Villa

Julian David Ramírez Rodríguez

Juan Andrés Gaviria

26 de mayo de 2026

Responsabilidades:

- **David Salazar** — Modelado matemático y derivación de constantes
- **Julian Ramírez** — Implementación numérica (Euler, Euler Mejorado, RK4) y análisis de error
- **Juan Gaviria** — Simulación de casos, animaciones y visualización

Resumen

Se modela la dinámica del thermal throttling en procesadores de computadores de propósito general bajo cargas de trabajo como un sistema de tres ecuaciones diferenciales no lineales que relacionan temperatura, potencia y throughput efectivo. La función de throttling logística introduce una no linealidad irreducible que impide la solución analítica y divide el comportamiento en tres regímenes: convergencia estable, oscilación sostenida con degradación parcial y throttling profundo. Se implementan Euler, Euler Mejorado y RK4 desde cero, con análisis de error comparativo y validación contra RK45. Se simulan diez casos de prueba variando sistemáticamente los parámetros del modelo para explorar los distintos regímenes dinámicos del sistema. Los resultados se animan para ilustrar las trayectorias en el espacio de estados.

1. Introducción

Cuando un procesador trabaja de forma sostenida, tres fuerzas compiten entre sí: el sistema operativo exige más potencia para maximizar el rendimiento, el efecto Joule convierte esa potencia en calor, y el firmware recorta la frecuencia para evitar daño físico irreversible en el silicio. El resultado es el *thermal throttling*: una caída brusca del rendimiento que se activa de forma no lineal conforme la temperatura se acerca al umbral crítico del chip. El fenómeno es relevante en servidores, estaciones de renderización y cualquier equipo sometido a cargas continuas, pues determina si el sistema puede sostener su carga de trabajo o entra en ciclos de degradación permanentes.

El sistema no admite solución analítica porque las tres variables —temperatura, throughput y potencia— están aco-

pladas de forma bidireccional: la potencia calienta al procesador, el calor activa el throttling, el throttling reduce el throughput, y el déficit de throughput lleva al sistema operativo a demandar más potencia. Este bucle de retroalimentación, combinado con la función sigmoide que modela la activación gradual del firmware, impide separar las ecuaciones e integrarlas de forma independiente.

1.1. Objetivos

El objetivo principal es modelar y simular mediante métodos numéricos la dinámica térmica y de rendimiento de un procesador bajo carga sostenida, identificando las condiciones que determinan si el sistema converge a un equilibrio estable o entra en régimen oscilatorio.

Como objetivos específicos:

1. Implementar Euler, Euler Mejorado y RK4 desde cero para el sistema vectorial de tres estados.
2. Comparar la convergencia de los tres métodos variando el tamaño de paso h y validar contra el solucionador de referencia RK45.
3. Simular diez casos variando sistemáticamente R_{obj} , α , k_2 , γ , β y k_{sig} para explorar los distintos regímenes del sistema.
4. Animar las trayectorias en el espacio de estados para ilustrar la transición entre regímenes.

1.2. Estructura del informe

La Sección 2 presenta el modelo matemático y justifica las constantes físicas. La Sección 3 describe los métodos numéricos, el análisis de error y la validación. La Sección 4 detalla los diez casos de simulación y sus interpretaciones. La Sección 5 sintetiza los hallazgos y reflexiona sobre el trabajo.

2. Metodología

2.1. Modelo matemático

Las variables de estado son $T(t)$ [°C], $P(t)$ [W] y $R(t)$ [GFLOPS].

Ecuación 1: Dinámica Térmica

$$\frac{dT}{dt} = k_1 P(t) - k_2 (T(t) - T_{\text{amb}}) \quad (1)$$

Basada en el Modelo de Capacitancia Térmica Concentrada ($C_{\text{th}} dT/dt = \dot{E}_{\text{in}} - \dot{E}_{\text{out}}$). El término $k_1 P$ es la tasa de calentamiento por efecto Joule; $-k_2(T - T_{\text{amb}})$ es la disipación por convección hacia el ambiente. [1]

Ecuación 2: Dinámica de Rendimiento

$$\frac{dR}{dt} = \gamma P(t) - \delta R(t) \quad (2)$$

Derivada del modelo de fluidos en teoría de colas ($\dot{x} = \lambda - \mu x$). El término γP es la aceleración computacional; $-\delta R$ es la fricción por saturación de buses (Modelo Roofline: el cuello de botella es el enlace más lento). [2, 6]

Ecuación 3: Control de Potencia con Throttling

$$\frac{dP}{dt} = \alpha (R_{\text{obj}} - R(t)) - \frac{\beta}{1 + e^{-k_{\text{sig}}(T(t) - T_{\text{crit}})}} \quad (3)$$

El término $\alpha(R_{\text{obj}} - R)$ es la demanda del SO (controlador proporcional [3]): α alto corresponde al perfil “Máximo rendimiento”. [4, 5]

2.2. Constantes del modelo

Cada constante se deriva como el producto de una magnitud de hardware medible y un factor de ajuste dimensional:

Cuadro 1: Constantes derivadas del hardware (equipo simulado)

Constante	Expresión	Valor
k_1	$0,0024 \cdot f_p \cdot N_c$	$0,0518 \text{ } ^\circ\text{C s}^{-1} \text{W}^{-1}$
k_2	$0,556 \cdot \Phi_{\text{aire}} \cdot H_d$	$0,1001 \text{ s}^{-1}$
γ	$0,0231 \cdot N_h \cdot f_p$	$0,9979 \text{ GFLOPS s}^{-2} \text{W}^{-1}$
δ	$336 / \min(v_{\text{ram}}, v_{\text{bus}} \times 10^3)$	$0,105 \text{ s}^{-1}$
α	$0,8 \cdot N_h / N_c$	1,6
β	calibración directa	12,0 W
k_{sig}	calibración directa	$0,5 \text{ } ^\circ\text{C}^{-1}$

Hardware base: $N_c = 6$, $N_h = 12$, $f_p = 3,6 \text{ GHz}$, $v_{\text{ram}} = 3200 \text{ MT/s}$, $v_{\text{bus}} = 16,0 \text{ GT/s}$, $\Phi_{\text{aire}} = 45,0 \text{ CFM}$, $H_d = 0,004 \text{ W/}^\circ\text{C}$.

Cada factor de ajuste convierte las unidades del hardware (GHz, CFM, MT/s) a las unidades que requieren las EDOs. Sus valores se fijaron por análisis dimensional y calibración sobre el caso base ($T^* = 79,5 \text{ } ^\circ\text{C}$, $P^* = 105,2 \text{ W}$):

- $c_{k_1} = 0,0024 \text{ } [^\circ\text{C}/(\text{GHz} \cdot \text{núcleo} \cdot \text{W} \cdot \text{s})]$: convierte la disipación de calor por efecto Joule a la tasa dT/dt correcta

para un chip de silicio de gama media-alta. Fue ajustado para que el equilibrio térmico en el caso base quede en el rango físico $70\text{--}85 \text{ } ^\circ\text{C}$.

- $c_{k_2} = 0,556 \text{ [adim., escala } \Phi_{\text{aire}} \cdot H_d \text{ a } \text{s}^{-1}]$: convierte el producto $\text{CFM} \times (\text{W}/^\circ\text{C})$ a la constante de tiempo de disipación del modelo concentrado. Su valor refleja la eficiencia media de disipadores de torre de bajo-medio costo frente a las condiciones del caso base.
- $c_\gamma = 0,0231 \text{ [GFLOPS}/(\text{hilo} \cdot \text{GHz} \cdot \text{W} \cdot \text{s})]$: convierte la capacidad de cómputo ($\text{hilos} \times \text{GHz}$) y la potencia a la aceleración de throughput dR/dt . Calibrado para que $\gamma P^* \approx \delta R^*$ en el equilibrio, garantizando consistencia dimensional.
- $c_\delta = 336,0 \text{ [s/MT} \cdot \text{s}^{-1}]$: convierte la velocidad de memoria (MT/s) al coeficiente de fricción $\delta \text{ [s}^{-1}]$. Representa la relación entre el ancho de banda disponible y el throughput que el sistema puede sostener; el denominador toma el canal más lento (Modelo Roofline), con v_{bus} escalado a MT/s para comparar en la misma unidad.
- $c_\alpha = 0,8 \text{ [adim., escala } N_h/N_c \text{ a agresividad del gobernador}]$: pondera el ratio hilos/núcleos con la sensibilidad típica del planificador del SO en modo rendimiento. Un valor mayor representaría un perfil de energía más agresivo (“Ultra Performance”), un valor menor, un perfil conservador (“Power Saver”).

β y k_{sig} se calibran directamente porque dependen del comportamiento interno del firmware del fabricante, que no es público.

2.3. Análisis de puntos de equilibrio

Igualando las tres derivadas a cero con throttling despreciable ($T^* \ll T_{\text{crit}}$):

$$P^* = \frac{\delta R_{\text{obj}}}{\gamma} = 105,2 \text{ W}, \quad T^* = T_{\text{amb}} + \frac{k_1 \delta R_{\text{obj}}}{k_2 \gamma} = 79,5 \text{ } ^\circ\text{C} \quad (4)$$

Estabilidad. Si la temperatura sube ligeramente por encima de T^* —digamos, el sistema se enfría solo, sin necesitar intervención externa. Si el throughput cae momentáneamente, el SO detecta el déficit $R_{\text{obj}} - R > 0$, inyecta más potencia y restaura el rendimiento. El equilibrio funciona como un atractor: cualquier pequeña perturbación se amortigua y el sistema vuelve al punto de operación óptimo.

Régimen oscilatorio. Cuando la condición se viola, el equilibrio teórico T^* supera el umbral físico T_{crit} , lo que desencadena un ciclo sin salida: el firmware recorta la potencia para bajar la temperatura, el throughput cae por debajo del objetivo, el sistema operativo responde aumentando la potencia, y la temperatura vuelve a subir. El sistema no colapsa —sigue funcionando— pero nunca converge: oscila indefinidamente con un déficit de rendimiento proporcional a cuánto se viola la condición.

3. Metodología Numérica

3.1. Implementación de los tres métodos

El sistema $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$, $\mathbf{x} = (T, R, P)^\top$, se resuelve con tres métodos programados desde cero sobre la misma interfaz $\mathbf{f}$.

Euler explícito: $\mathbf{x}_{n+1} = \mathbf{x}_n + h \mathbf{f}(\mathbf{x}_n)$

Euler Mejorado (Heun): $\mathbf{x}^* = \mathbf{x}_n + h \mathbf{f}(\mathbf{x}_n)$, $\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{2}(\mathbf{f}(\mathbf{x}_n) + \mathbf{f}(\mathbf{x}^*))$

RK4:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (5)$$

con $\mathbf{k}_i$ las cuatro pendientes estándar.

Condición de frontera. $P(t)$ puede volverse negativa bajo throttling máximo sostenido, lo cual carece de sentido físico. Al final de cada paso: $P_{n+1} \leftarrow \max(P_{\min}, P_{n+1})$, con $P_{\min} = 10 \text{ W}$.

3.2. Análisis de error y validación

El error relativo global respecto a `scipy.integrate.solve_ivp` (RK45, `max_step= 0,01 s`) se define como:

$$E_{\text{rel}}(h) = \max_{t_i} \frac{\|\mathbf{x}_{\text{aprox}}(t_i) - \mathbf{x}_{\text{ref}}(t_i)\|}{\|\mathbf{x}_{\text{ref}}(t_i)\|} \quad (6)$$

Cuadro 2: $E_{\text{rel}}(h)$ por método y tamaño de paso

h (s)	Euler	Euler Mej.	RK4
1.00	21.8566	7.1936	4.7629
0.50	9.9628	5.1128	4.9795
0.10	5.9472	5.2857	5.2899
0.05	5.6031	5.2921	5.2930
0.01	5.3687	5.3093	5.3093

Los valores de E_{rel} se expresan en la escala natural del vector de estado ($T \approx 80^\circ\text{C}$, $R \approx 1000 \text{ GFLOPS}$, $P \approx 105 \text{ W}$): magnitudes de ≈ 5 corresponden a errores absolutos pequeños en proporción al estado. El error se acumula principalmente en la fase transitoria, no en el estado estacionario.

Euler muestra sensibilidad cuadrática al paso: el error se cuadruplica de $h = 0,01$ a $h = 1,0$, coherente con $O(h)$. Con $h = 1,0$ produce estados físicamente incorrectos ($P = 542,9 \text{ W}$), indicando inestabilidad en el transitorio. **Euler Mejorado** reduce el error a la mitad para $h = 1,0$ ($O(h^2)$) y converge con $h = 0,5 \text{ s}$. **RK4** es el más robusto: alcanza el equilibrio correcto incluso con $h = 1,0 \text{ s}$ ($O(h^4)$).

Comparación en $t = 300 \text{ s}$ con $h = 0,05 \text{ s}$: el error residual de Euler en P es de 2.49 W ; RK4 y Euler Mejorado lo reducen a 0.0001 W . La sigmoide introduce un cambio brusco en $\dot{P}$ cerca de T_{crit} que Euler no capta con suficiente fidelidad incluso con pasos pequeños.

Convergencia y eficiencia. RK4 y Euler Mejorado son equivalentes en precisión para $h \leq 0,1 \text{ s}$, pero RK4 requiere cuatro evaluaciones de $\mathbf{f}$ por paso frente a las dos de Euler Mejorado. Dado que $\mathbf{f}$ es analítica y barata de evaluar, el costo adicional es despreciable; RK4 es la elección natural por su mayor orden y estabilidad.

3.3. Herramientas y proceso

Python 3.x con NumPy (vectorial), SciPy (RK45 referencia), Matplotlib (visualización) y p5.js (animaciones interactivas). Los tres métodos se programaron desde cero con interfaz idéntica sobre $\mathbf{f}(\mathbf{x})$, permitiendo intercambiarlos en el bucle de simulación sin modificar el resto del código.

3.4. Uso de IA

{ia} Consulta de conceptos teóricos: Modelo Roofline, Capacitancia Concentrada, funciones de barrera.

{ia} Identificación de referencias bibliográficas relevantes.

{ia} Revisión de la verificación dimensional de constantes tras análisis propio.

{ia} Sugerencia de condiciones iniciales con comportamientos interesantes, exploradas después de forma propia.

No se empleó IA para generar código, producir imágenes o animaciones. Se utilizó como apoyo en dos tareas adicionales: la verificación de consistencia dimensional de los factores de ajuste c_{k1} , c_{k2} , c_γ , c_δ y c_α , cuyas unidades de hardware (GHz, CFM, MT/s) no son directamente compatibles con las unidades de las EDOs, y como herramienta en el análisis comparativo de los métodos numéricos, revisando que las conclusiones sobre convergencia y error fueran consistentes con la teoría de métodos de paso fijo.

4. Resultados y Discusión

4.1. Caso base y validación

Condiciones iniciales: $T(0) = 45^\circ\text{C}$, $R(0) = 0 \text{ GFLOPS}$, $P(0) = 15 \text{ W}$; $T_{\text{amb}} = 25^\circ\text{C}$, $T_{\text{crit}} = 90^\circ\text{C}$, $R_{\text{obj}} = 1000 \text{ GFLOPS}$. RK4, Euler Mejorado y RK45 convergen al mismo equilibrio ($T^* = 79,5^\circ\text{C}$, $R^* = 1000,0 \text{ GFLOPS}$, $P^* = 105,2 \text{ W}$), validando la implementación. Euler con $h = 0,1$ aún no converge ($T = 78,3^\circ\text{C}$, $R = 976,3 \text{ GFLOPS}$), evidenciando su sensibilidad al paso.

4.2. Diez casos de simulación

Se ejecutaron diez casos con RK4 ($h = 0,05 \text{ s}$, $t = 300 \text{ s}$) variando sistemáticamente los parámetros del modelo. La Tabla 3 resume los equilibrios teóricos y resultados simulados.

Cuadro 3: Resultados de los diez casos de simulación (RK4, $h = 0,05$ s, $t = 300$ s)

#	Descripción	R_{obj}	α	k_2	γ	β/k_{sig}	$T_{\text{teórico}}^*$	T_{sim}	R_{sim}	Régimen
1	Base estable	1000	1.6	0.1001	0.9979	12/0.5	79.5°C	79.5°C	1000.0	Estable ✓
2	Alta carga	1500	1.6	0.1001	0.9979	12/0.5	106.8°C	106.3°C	1492.5	Oscilatorio
3	Undervolting	1000	0.4	0.1001	0.9979	12/0.5	79.5°C	79.5°C	999.8	Estable ✓
4	Sobrevoltaje overshoot	1000	3.2	0.1001	0.9979	12/0.5	79.5°C	79.5°C	1000.0	Estable ✓
5	Mejor disipación	1000	1.6	0.2002	0.9979	12/0.5	52.3°C	52.3°C	1000.0	Estable ✓
6	Mala disipación	1000	1.6	0.0400	0.9979	12/0.5	161.3°C	160.2°C	992.5	Throttling prof.
7	CPU eficiente	1000	1.6	0.1001	1.9958	12/0.5	52.3°C	52.3°C	1000.0	Estable ✓
8	CPU ineficiente	1000	1.6	0.1001	0.4990	12/0.5	134.0°C	133.2°C	992.5	Throttling prof.
9	Throttling suave	1300	1.6	0.1001	0.9979	5/0.2	95.9°C	95.7°C	1297.6	Oscil. leve
10	Throttling agresivo	1300	1.6	0.1001	0.9979	25/2.0	95.9°C	95.0°C	1284.4	Oscilatorio

Casos estables (1, 3, 4, 5, 7)

Los cinco casos con $T^* < T_{\text{crit}}$ convergen al equilibrio teórico con error inferior a 0.02 GFLOPS, confirmando la condición (??).

Casos 3 y 4 ($\alpha = 0,4$ y $\alpha = 3,2$): variar la agresividad del gobernador entre un cuarto y el doble del valor base no altera el equilibrio final (diferencia $< 0,2$ GFLOPS), solo la velocidad de convergencia y la magnitud del overshoot en la fase transitoria. α controla la *dinámica*, no el estado estacionario.

Caso 5 ($k_2 \times 2$): duplicar la conductancia térmica desplaza T^* de 79.5°C a 52.3°C (-34%) sin pérdida de throughput. El margen térmico pasa de 10.5°C a 37.7°C: mayor retorno de cualquier intervención de hardware.

Caso 7 ($\gamma \times 2$): doble eficiencia computacional produce el mismo throughput con la mitad de potencia (52.6 W vs 105.2 W) y la misma reducción de temperatura. Modela el impacto de un salto generacional en la arquitectura.

Casos inestables (2, 6, 8, 9, 10)

Caso 2 ($R_{\text{obj}} = 1500$): $T^* = 106,8^\circ\text{C}$ supera T_{crit} . El firmware limita continuamente la potencia; el sistema opera con déficit sostenido de 7.5 GFLOPS.

Caso 6 ($k_2 \times 0,4$): conductancia reducida al 40 % lleva T^* a 161.3°C. Throttling profundo continuo con oscilaciones amplias en R y P . Modela un disipador deteriorado.

Caso 8 ($\gamma \times 0,5$): la mitad de eficiencia exige el doble de potencia (208.9 W), llevando el sistema a un régimen idéntico al caso 6. Representa hardware de arquitectura antigua.

Casos 9 vs 10 (misma carga $R_{\text{obj}} = 1300$, distinto firmware): el throttling agresivo ($\beta = 25$, $k_{\text{sig}} = 2,0$) reduce la temperatura 0.7°C más que el suave, pero amplifica las oscilaciones de throughput en un factor de 6 (déficit de 15.6 vs 2.4 GFLOPS). El firmware más agresivo protege la temperatura a costa de mayor inestabilidad en el rendimiento.

Síntesis de regímenes

Los diez casos identifican tres regímenes dinámicos: (1) **convergencia estable** cuando $T^* < T_{\text{crit}}$; las inter-

venciones sobre k_2 y γ desplazan T^* sin alterar R^* ; (2) **oscilación sostenida** cuando T^* supera levemente T_{crit} , con déficit de throughput proporcional a la severidad del throttling; y (3) **throttling profundo** cuando $T^* \gg T_{\text{crit}}$, con cortes máximos persistentes y operación cerca del límite físico.

4.3. Animaciones

1. **Evolución temporal** con 8 parámetros ajustables en vivo (deslizadores de $k_1, k_2, \gamma, \delta, \alpha, \beta, k_{\text{sig}}, R_{\text{obj}}$), ventilador automático y barra de throttling $\sigma(t)$: <https://editor.p5js.org/Alex-sklx0/sketches/7KqpJfRie> <https://editor.p5js.org/Alex-sklx0/sketches/YZkcDwXYb>
2. **Activación del throttling**: dos paneles $T(t)$ y $\sigma(t)$; control de calor externo para empujar $T \rightarrow T_{\text{crit}}$ y comparar pendiente suave vs corte abrupto: <https://editor.p5js.org/Alex-sklx0/sketches/fQWku9xab>
3. **Estable vs oscilatorio**: comparación lado a lado de $T(t)$ con k_2 nominal (estable) y $k_2 \times$ multiplicador ajustable; umbral de transición visible en $\approx 0,7 \times$: https://editor.p5js.org/Alex-sklx0/sketches/g4Q6W-0_4

5. Conclusiones

5.1. Síntesis de resultados

El resultado central es la condición de viabilidad térmica (??), que predice el régimen del sistema —estable u oscilatorio— sin integrar ninguna ecuación. Los diez casos la validan: todos los que la violan producen oscilación o throttling profundo; todos los que la cumplen convergen al equilibrio con error $< 0,02$ GFLOPS.

Las intervenciones más efectivas por impacto sobre T^* : (1) mejorar la conductancia k_2 (caso 5: -34% en T^* , margen de 37.7°C, sin costo en R^*); (2) usar hardware con mayor eficiencia γ (caso 7: misma T^* que caso 5, con la mitad de potencia); (3) ajustar α (casos 3 y 4: solo modifica

el transitorio, no el equilibrio); (4) configurar β y k_{sig} del firmware (casos 9 y 10: trade-off entre estabilidad térmica y estabilidad del throughput).

5.2. Implicaciones prácticas

Dado el hardware de un equipo, la condición (??) permite calcular el $R_{\text{obj}}^{\text{max}}$ sostenible sin throttling, antes de ejecutar ninguna carga. Esto permite dimensionar correctamente servidores y estaciones de cómputo sin llevar el hardware al punto de fallo. El modelo evidencia además que el throttling no es el problema original sino la consecuencia de un sistema mal dimensionado: la causa raíz es la violación de la viabilidad térmica.

5.3. Sobre los métodos numéricos

RK4 es la elección correcta para este sistema: su orden $O(h^4)$ alcanza el equilibrio con $h = 1,0\text{s}$ donde Euler es inestable, y elimina el error residual de 2.49 W en P que Euler mantiene incluso con $h = 0,05\text{s}$. La validación contra RK45 confirmó que los tres métodos implementados desde cero son correctos y convergen al mismo estado estacionario.

5.4. Reflexión sobre el trabajo en equipo

Tres dificultades técnicas marcaron el proyecto. La **calibración dimensional**: garantizar consistencia de unidades entre variables de hardware (GHz, MT/s, CFM) y las EDOs requirió análisis dimensional iterativo y múltiples revisiones. La **condición de frontera en RK4**: aplicarla en los estados intermedios $\mathbf{k}_i$ introduce una discontinuidad que altera la trayectoria; fue necesario decidir conscientemente dónde aplicarla para mantener la física sin comprometer la estabilidad del integrador. El **recálculo en tiempo real en p5.js**: sincronizar la reintegración completa del sistema con la actualización visual al mover un deslizador exigió optimizar el bucle numérico para no bloquear el render.

El aprendizaje más valioso fue constatar que el análisis de equilibrio analítico es más poderoso que la simulación bruta: la condición (??) predice el régimen antes de integrar una sola ecuación.

Repositorio

Código disponible en:
<https://github.com/Alex-sk1x0/Modelo-Dinamico-de-Estabilidad-Termica-y-Rendimiento-en-Computadores-de-Proposito-General>

Referencias

- [1] Incropera, F.P. & DeWitt, D.P. (2002). *Fundamentos de transferencia de calor y masa*. Wiley.
- [2] Kleinrock, L. (1976). *Queueing Systems, Vol. II*. Wiley-Interscience.
- [3] Ogata, K. (2010). *Ingeniería de control moderna*. Pearson.
- [4] Brooks, D. & Martonosi, M. (2001). Dynamic Thermal Management for High-Performance Microprocessors. *HPCA 2001*.
- [5] Nocedal, J. & Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer.
- [6] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline model. *Commun. ACM*, 52(4).
- [7] Hennessy, J.L. & Patterson, D.A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- [8] Ames, A.D. et al. (2019). Control Barrier Functions. *18th European Control Conference*.
- [9] Rao, S.S. (2019). *Engineering Optimization* (5th ed.). Wiley.